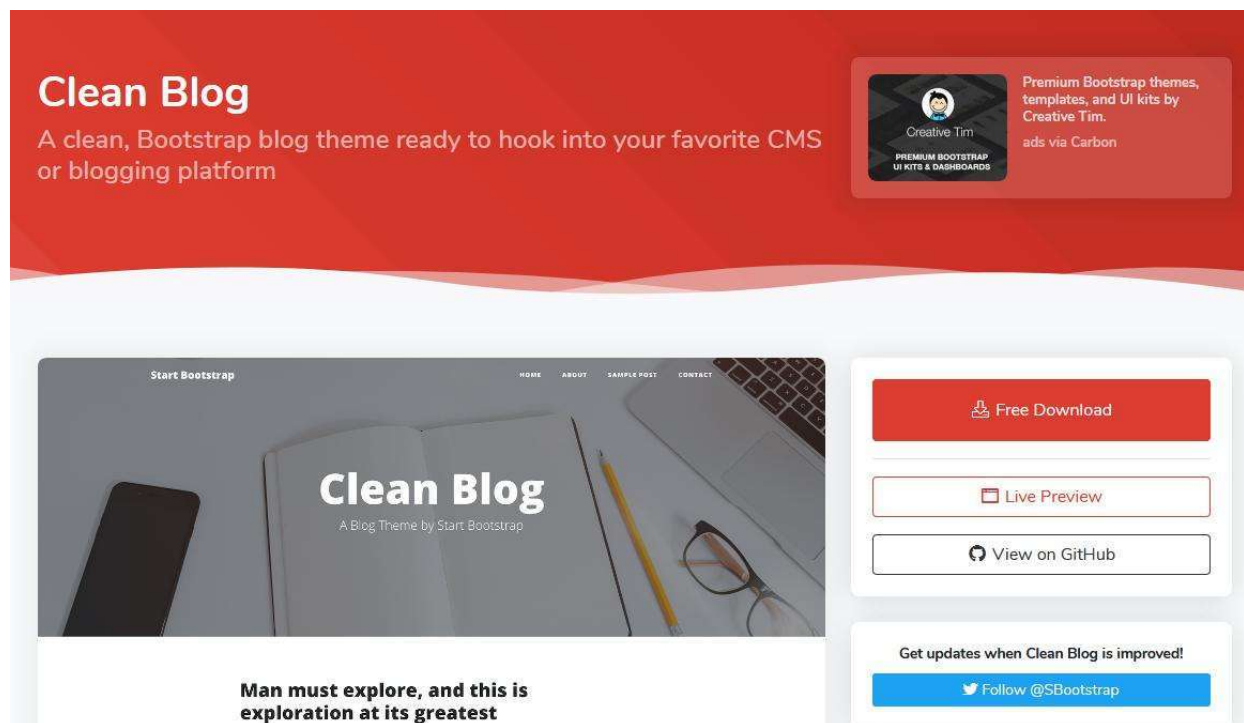


Bước đầu tạo web app với Express

Vì mục tiêu của cuốn sách này không tập trung vào hướng dẫn các bạn thiết kế website bằng HTML, CSS. Do vậy, chúng ta sẽ sử dụng một template có sẵn để thực hành Node.js + ExpressJS. Để tìm một template, bạn vào trang startbootstrap.com và lựa chọn một template ưng ý nhất.

Tải web template

Trong cuốn sách này, mình chọn clean blog template - Hình 3.1 (<https://startbootstrap.com/themes/clean-blog/>)



Hình 3.1 Giao diện một web template

Bạn tải miễn phí template này về, và giải nén vào một thư mục trong máy tính.

Bước tiếp theo, chúng ta sẽ khởi tạo dự án Node.js mới bằng câu lệnh: `npm init`

(Bạn cứ trả lời "Yes" cho các câu hỏi mà NPM hỏi để tạo ra *package.json*). Tiếp theo thì mình cài đặt express cho dự án.

```
$ npm install express
```

Ok, giờ bạn tạo file *index.js* trong thư mục gốc của dự án, và viết đoạn code tạo server đơn giản như sau (Nếu bạn quên thì quay lại đọc phần 2 - mình đã giới thiệu cách tạo server đơn giản):

```
const express = require('express')

const app = new express()
app.listen(4000, () => {
  console.log('App listening on port 4000')
})
```

Để chạy server, bạn gõ lệnh sau vào terminal:

```
$ node index.js
```

Tự động khởi động server mỗi khi thay đổi mã nguồn

Mặc định, để kiểm tra kết quả mỗi khi bạn thay đổi mã nguồn, bạn cần phải stop/start lại server, có như vậy đoạn code mới có hiệu lực. Nếu bạn muốn công đoạn này được thực hiện tự động, chỉ cần mỗi khi mã nguồn có thay đổi thì server sẽ tự động khởi động lại. Để làm được điều này, chúng ta cần phải cài đặt thêm thư viện theo dõi mã nguồn. Có nhiều thư viện làm điều như: [forever](#), [nodemon](#)...

Trong cuốn sách này, chúng ta sẽ sử dụng nodemon. Để cài đặt nodemon cho dự án, từ terminal:

```
$ npm install nodemon --save-dev
```

Để mình giải thích qua một chút về các tham số trong câu lệnh trên:

- **Tham số --save:** mục đích là sau khi cài đặt xong vào thư mục *node_modules* thì cũng thêm thông tin vào mục *dependencies* trong *package.json*. Với việc thêm vào *dependencies*, sau này người khác muốn cài đặt các thư viện cần thiết cho dự án thì cần gõ: `npm install` là nó tự cài đặt tất cả *dependencies* được liệt kê trong *package.json*
- **Tham số -dev:** Có những thư viện chỉ dùng để hỗ trợ phát triển dự án, còn khi triển khai thật ra thị trường thì không cần. Đó là lý do mà phần *dependencies* của *package.json* được chia làm 2 phần: "*dependencies*" và "*devDependencies*". Tham số `-dev` là để thêm vào "*devDependencies*".

Npm start

Ở phần trên của cuốn sách, mình có hướng dẫn các bạn khởi động server bằng lệnh:
`node <tên_file_js>`

Tuy nhiên, với cách làm này, nếu bạn cần phải tùy biến lệnh khởi động server thì sẽ rất khó nhớ lệnh cho những lần thực hiện sau. Do đó, để đơn giản và rút gọn câu lệnh, chúng ta có thể sử dụng câu lệnh: `npm start`.

Các bạn mở `package.json`, rồi tìm đến thẻ `scripts` và sửa như sau:

```
"scripts": {  
  "start": "nodemon index.js"  
}
```

Từ giờ, mỗi lần khởi động server, bạn chỉ cần gõ lệnh:

```
$ npm start
```

Với cách làm này, bạn sẽ không còn phải quan tâm file chính của dự án là file nào, `index.js` hay `app.js`... chỉ đơn giản gõ `npm start` là được.

Tạo thư mục public chứa tệp static

Như mình đã giải thích ở phần trước, các tệp static được hiểu là những tệp tạo giao diện, tương tác trực tiếp với người dùng, truy xuất không cần qua router. Những tệp này gồm: ảnh, css, html, js.

Về lý thuyết thì mình đã hướng dẫn ở phần trước cuốn sách rồi, giờ chúng ta bắt tay vào thực hành tạo những tệp static cho blog dự án của chúng ta.

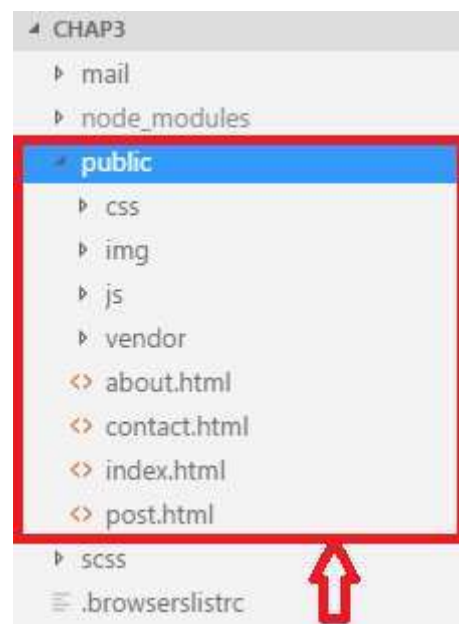
```
const express = require('express')  
const app = new express()  
  
//Đăng ký thư mục public.  
app.use(express.static('public'))  
  
app.listen(4000, () => {  
  console.log('App listening on port 4000')  
})
```

Bởi vì chúng ta đang sử dụng template có sẵn nên đã có những tệp html trước rồi, không cần phải tạo nữa. Giờ bạn chỉ cần chuyển những tệp html vào thư mục *public* này. Hiện tại, trong template có những tệp html sau sẽ được chuyển vào thư mục *public*.

index.html
about.html
contact.html
post.html

Tiếp theo, chúng ta sẽ chuyển tất cả những thư mục *css*, *img*, *js*, *vendor* vào trong thư mục *public*.

OK, giờ bạn sẽ thấy cấu trúc thư mục dự án mới sẽ như này.



Hình 3.2: thư mục *public* trong dự án

Bạn thử mở một tệp html lên xem mã nguồn, bạn sẽ biết tạo chúng ta lại chuyển những thư mục *css*, *js*... vào thư mục *public*.

Ví dụ, với tệp *index.html*, bạn có thấy các thẻ link tới thư mục *vendor*, *css*... (mình có bôi đậm trong code đó).

```
<head>  
<title>Clean Blog - Start Bootstrap Theme</title>
```

```
<!-- Bootstrap core CSS -->
```

```
<link href="vendor/bootstrap/css/bootstrap.min.css" rel="stylesheet">

<!-- Custom fonts for this template -->
<link href="vendor/fontawesome-free/css/all.min.css" rel="stylesheet"
type="text/css">
...
<!-- Custom styles for this template -->
<link href="css/clean-blog.min.css" rel="stylesheet">

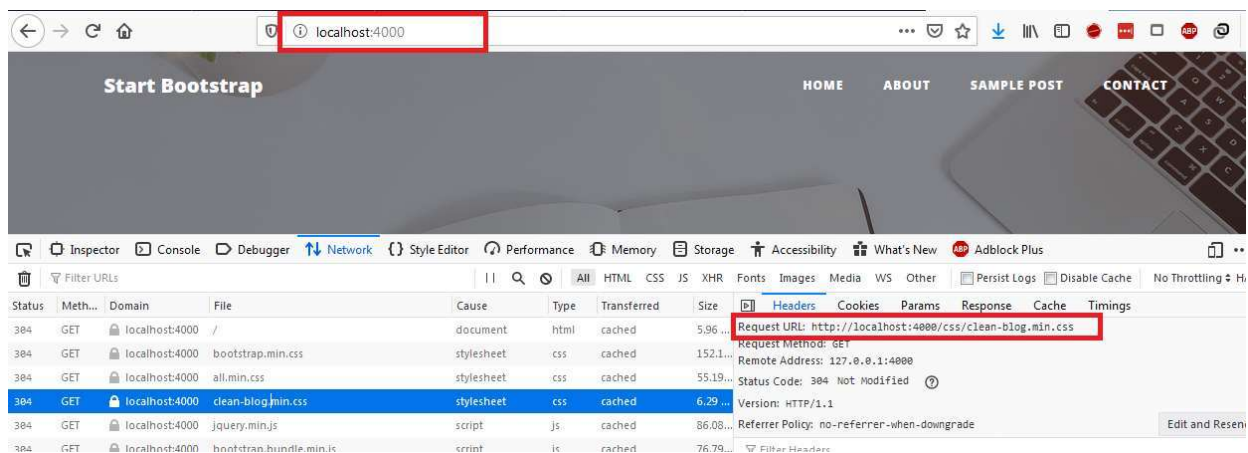
</head>
```

Với những kiểu tương đối như: **css/clean-blog.min.css** thì khi trình duyệt đọc tệp html này sẽ tự động nối thêm domain vào để tạo thành một link hoàn chỉnh:

`http://<domain>/css/clean-blog.min.css`

Để mình chỉ có bạn. Giờ bạn chạy server bằng lệnh: `$ npm start`

Sau đó vào trình duyệt kiểm tra: `http://localhost:4000`



Hình 3.3: Kiểm tra link static resource trên trình duyệt

Nếu bạn click vào các link trên giao diện như: About, Same post, Contact, nó sẽ chuyển đến đúng trang đích. Tuy nhiên, hiện giờ trang web của chúng ta chỉ thuần là trang web tĩnh, các file html liên kết với nhau mà chưa hề xử lý gì cả.

Việc tiếp theo, chúng ta sẽ xử lý việc chuyển trang (about, sample post, contact) thông qua route.

Tạo Page routes

Như mình đã nói ở trên, hiện tại chúng ta mới chỉ đơn giản là phản hồi nguyên một tệp html xuống cho client từ thư mục public. Có một nhược điểm của thư mục public là ai cũng có thể truy cập được, server hầu như chỉ biết trả về cho client tệp mà nó yêu cầu, không thể xử lý gì được nữa.

Để có thể xử lý được logic trước khi trả về cho client, ví dụ như cần xác thực request xem request này có hợp lệ không... thì bắt buộc bạn phải sử dụng routes.

Để thực hành tạo routes, chúng ta tạo một thư mục đặt tên là pages. Sau đó lại copy tệp *index.html* vào đó.

Sau đó, chúng ta mở tệp *index.js* và tạo route như sau:

```
const express = require('express')
const app = new express()
const path = require('path')
...
app.get('/', (request, response) =>{
  response.sendFile(path.resolve(__dirname, 'pages/index.html'))
})
```

Giờ cứ có một request tới trang chủ "/", thì trang *index.html* sẽ được trả về.

Tổng kết

Như vậy, qua phần 3 của cuốn sách này, chúng ta đã bắt đầu tạo được một trang web từ một template có sẵn. Biết cách sử dụng nodemon để tự động phát hiện sự thay đổi trong mã nguồn để khởi động lại node server, giúp đoạn code thay đổi có hiệu lực. Cuối cùng thì bạn đã biết cách tạo route để giúp ứng dụng có thể chuyển qua lại giữa các trang (page).

Các bạn có thể tham khảo mã nguồn của phần này tại đây:

<https://github.com/vntalking/nodejs-express-mongodb-co-ban/tree/master/chap3>

Nếu bạn có bất kỳ thắc mắc hoặc chỗ nào chưa hiểu, đừng ngại liên hệ với mình qua support@vntalking.com.